ARTI 502

Deep Learning: 1st Assignment

Training-Testing Neural Net with MNIST Dataset using PyTorch

Year 2026 Term 1

Group # 2

Group Members:

#	Member Name	Member Academic ID
1	Mohammed Jaarran	2230000129
2	Mohaiya Aldossary	2230004579
3	Sameer Almadani	2230001818
4	Omar Bodai	2240040094
5	Hussain Alabdurabalnabi	2230004638

Task 1: Install PyTorch

We installed PyTorch, torchvision and matplotlib with pip from inside the notebook, then imported torch and the other things we need later (torch.nn, torch.nn.functional and matplotlib). We printed the version to make sure PyTorch is loaded. The notebook is saved as "Assigment-1 DL".

```
[1]  # TASK – 1. Install PyTorch.
     import os
     os.environ["KMP_DUPLICATE_LIB_OK"] = "TRUE" ## << these two lines are added because my kernel crashes without it.
     !pip install torch torchvision matplotlib
     import torch
     import torch.nn as nn
     import torch.nn.functional as F
     ## ^^ Imported for Task 4.
     import matplotlib.pyplot as plt
     ## ^^ Imported for Task 3.


     print(torch.__version__)
```

```
...
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /Users/almadani/miniconda3/lib/python3.13/site-packages (from
sympy>=1.13.3->torch) (1.3.0)
Requirement already satisfied: MarkupSafe>=2.0 in /Users/almadani/miniconda3/lib/python3.13/site-packages (from jinja2-
>torch) (3.0.2)
2.14.0
```

Installing and loading PyTorch (pip output cut because it was already installed)

PyTorch 2.14.0 is installed and loaded with no errors.

Task 2: Download and Load the Dataset

We used torchvision.datasets.MNIST with download=True, so the dataset is downloaded directly from the code into the ./data folder. transforms.ToTensor() changes every image into a tensor and scales the pixels to be between 0 and 1. Then we made a DataLoader for the training set and one for the test set with batch size 32. The training loader is shuffled and the test loader is not.

```
[2]  # TASK – 2. Download and Load the Dataset.
     import torchvision
     import torchvision.transforms as transforms

     transform = transforms.ToTensor()
     ## ^^ converts images to tensors and scales pixels to [0, 1].

     train_set = torchvision.datasets.MNIST(root='./data', train=True, download=True, transform=transform)
     ## ^^ prepares training set.
     test_set = torchvision.datasets.MNIST(root='./data', train=False, download=True, transform=transform)
     ## ^^ prepares test set.

     train_loader = torch.utils.data.DataLoader(train_set, batch_size=32, shuffle=True)
     ## ^^ loads training set.
     test_loader = torch.utils.data.DataLoader(test_set, batch_size=32, shuffle=False)
     ## ^^ loads test set.

     print("Train images:", len(train_set))
     print("Test images:", len(test_set))
```

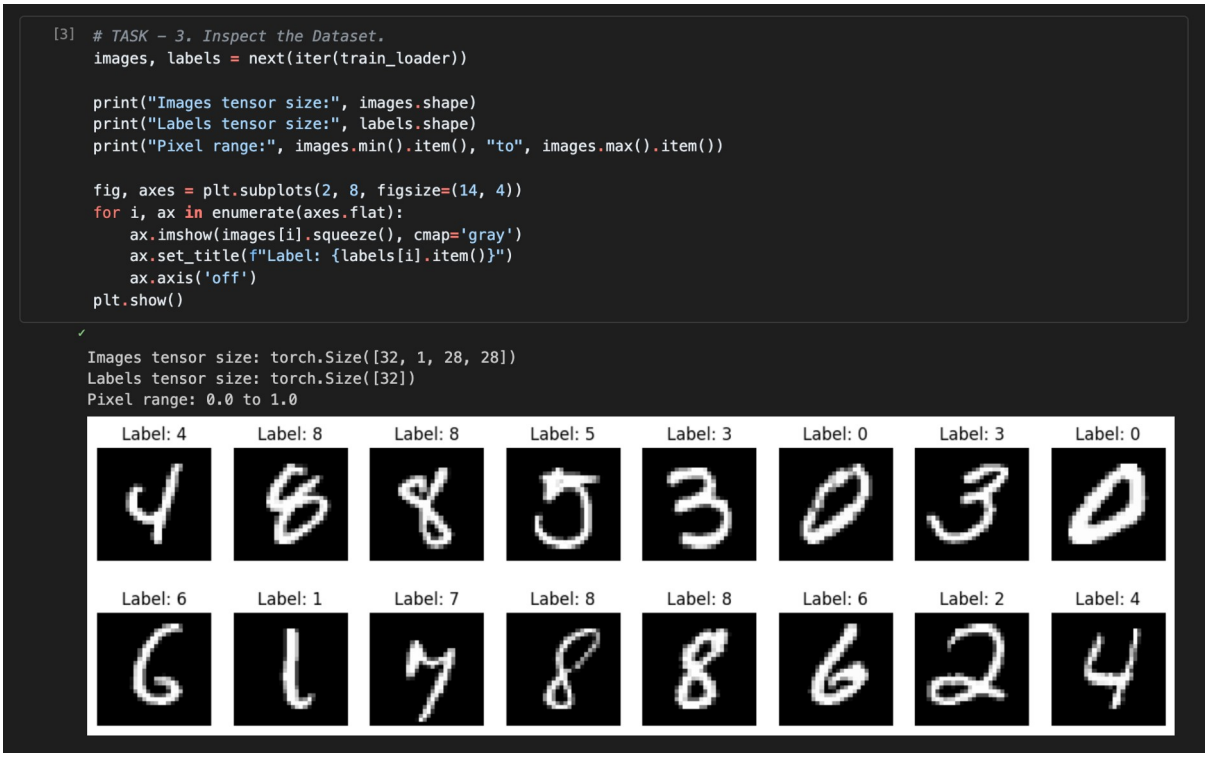
```
Train images: 60000
Test images: 10000
```

Dataset downloaded and loaded successfully

The training set has 60,000 images and the test set has 10,000 images. The assignment says 50,000 for training, but the MNIST training set in torchvision has 60,000 images, so we used all of them.

Task 3: Inspect the Dataset

We took one mini-batch from the train_loader to see how the dataloader loads the images into tensors, and then showed the first 16 images with matplotlib.



Images tensor size and visualization using matplotlib

1. Images tensor size

The images tensor size is [32, 1, 28, 28], which is N x C x H x W: 32 images in one batch, 1 channel because the images are grayscale, and 28 x 28 pixels. The labels tensor is [32], one label for each image. The pixel values go from 0.0 to 1.0 because of ToTensor.

2. Data images visualization using matplotlib

We used plt.subplots to show 16 images in 2 rows and put the label above each image. All labels match the digits in the images.

Task 4: Creating A Simple Network

SimpleNet has one hidden layer. fc1 is a fully connected layer (nn.Linear) that connects the 784 inputs (28 x 28) to 128 hidden neurons, and fc2 connects the 128 hidden neurons to the 10 outputs, one for each digit. We use ReLU on the hidden neurons as the nonlinearity.

nn.Linear expects a 2D input N x I, but our data is N x C x H x W, so in forward we use torch.flatten(x, 1) first to rearrange it.

```
[4] # TASK - 4. Creating A Simple Network.
    class SimpleNet(nn.Module):
        def __init__(self, input_size=784, hidden_size=128, num_classes=10):
                                  #28x28 ^^
            super().__init__()
            self.fc1 = nn.Linear(input_size, hidden_size)
            ## ^^ Layer 1
            self.fc2 = nn.Linear(hidden_size, num_classes)
            ## ^^ Layer 2

        def forward(self, x):
            x = torch.flatten(x, 1)   # (32, 1, 28, 28) -> (32, 784)
            x = F.relu(self.fc1(x))   # hidden layer + ReLU
            x = self.fc2(x)           # output layer (10 class scores)
            return x

    net = SimpleNet()
    print(net)

    # Verify against the requirement
    out = net(images)
    print("Input shape:", images.shape)
    print("Flattened shape:", torch.flatten(images, 1).shape)
    print("Output shape:", out.shape)

✓
SimpleNet(
  (fc1): Linear(in_features=784, out_features=128, bias=True)
  (fc2): Linear(in_features=128, out_features=10, bias=True)
)
Input shape: torch.Size([32, 1, 28, 28])
Flattened shape: torch.Size([32, 784])
Output shape: torch.Size([32, 10])
```

Source code of the simple network and verifying it

To verify the code with the requirement, we passed the batch from Task 3 into the network. After flatten the data goes from (32 x 1 x 28 x 28) to (32 x 784), which is exactly what is required, and the output is (32 x 10), 10 class scores for each image.

Task 5: Training Function

The train function takes the network and the train_loader as input, and it has optional parameters to control the training: epochs, lr and momentum.

- Loss: nn.CrossEntropyLoss, which applies softmax to the output and then minimizes the negative log likelihood.
- Optimizer: optim.SGD (stochastic gradient descent) with learning rate and momentum.
- For every mini-batch: zero_grad clears the old gradients, then the forward pass, then the loss, backward computes the gradients and step updates the weights.
- Every 200 batches it prints the epoch, batch number, current loss and accuracy.
- It returns all the mini-batch losses, plus the loss and accuracy of every epoch so we can plot them.

```
[5]  # TASK - 5. Training Function.
     import torch.optim as optim

     def train(net, train_loader, epochs=5, lr=0.01, momentum=0.9):

         criterion = nn.CrossEntropyLoss()
         optimizer = optim.SGD(net.parameters(), lr=lr, momentum=momentum)
         ## ^^ Pre determined from Tasks description.

         all_losses = []
         epoch_losses = []
         epoch_accs = []

         for epoch in range(epochs):
             running_loss = 0.0
             correct = 0
             total = 0

             for i, (inputs, labels) in enumerate(train_loader):
                 optimizer.zero_grad() #clears old gradients
                 outputs = net(inputs) #calls forward function
                 loss = criterion(outputs, labels) #computes the loss.
                 loss.backward() #computes gradients.
                 optimizer.step() #updates the weights.

                 all_losses.append(loss.item())
                 running_loss += loss.item()

                 _, predicted = torch.max(outputs, 1)
                 ## ^^^ picks the highest-scoring digit as the prediction, which is used for accuracy.

                 total += labels.size(0)
                 correct += (predicted == labels).sum().item()

                 if (i + 1) % 200 == 0:
                     print(f"Epoch {epoch+1}/{epochs}, Batch {i+1}, "
                           f"Loss: {loss.item():.4f}, Acc: {100*correct/total:.2f}%")
                     ## ^^ Epoch Debug Info for TASK - 6.

             epoch_losses.append(running_loss / len(train_loader))
             epoch_accs.append(100 * correct / total)
             print(f"--- Epoch {epoch+1} done: Loss {epoch_losses[-1]:.4f}, Acc {epoch_accs[-1]:.2f}%")

         return all_losses, epoch_losses, epoch_accs
```

Training function source code

Task 6: Training Network

We instantiated a new SimpleNet and trained it with our train function for 5 epochs, with learning rate 0.01 and momentum 0.9.

```
[6]  # TASK - 6. Training Network.

     net = SimpleNet()
     # ^^ create network from TASK - 4 class.

     all_losses, epoch_losses, epoch_accs = train(net, train_loader, epochs=5, lr=0.01, momentum=0.9)
     # ^^ Training the network and retrieving feedback from Train TASK - 5 function.

✓
Epoch 1/5, Batch 200, Loss: 0.5774, Acc: 72.33%
Epoch 1/5, Batch 400, Loss: 0.5708, Acc: 80.09%
Epoch 1/5, Batch 600, Loss: 0.1866, Acc: 83.22%
Epoch 1/5, Batch 800, Loss: 0.3428, Acc: 85.26%
Epoch 1/5, Batch 1000, Loss: 0.1215, Acc: 86.61%
Epoch 1/5, Batch 1200, Loss: 0.2404, Acc: 87.46%
Epoch 1/5, Batch 1400, Loss: 0.0761, Acc: 88.20%
Epoch 1/5, Batch 1600, Loss: 0.1006, Acc: 88.83%
Epoch 1/5, Batch 1800, Loss: 0.1759, Acc: 89.40%
--- Epoch 1 done: Loss 0.3637, Acc 89.56%
Epoch 2/5, Batch 200, Loss: 0.1872, Acc: 94.67%
Epoch 2/5, Batch 400, Loss: 0.0711, Acc: 94.84%
Epoch 2/5, Batch 600, Loss: 0.0797, Acc: 94.96%
Epoch 2/5, Batch 800, Loss: 0.1371, Acc: 95.05%
Epoch 2/5, Batch 1000, Loss: 0.2587, Acc: 95.06%
Epoch 2/5, Batch 1200, Loss: 0.0461, Acc: 95.09%
Epoch 2/5, Batch 1400, Loss: 0.0944, Acc: 95.17%
Epoch 2/5, Batch 1600, Loss: 0.0153, Acc: 95.20%
Epoch 2/5, Batch 1800, Loss: 0.1166, Acc: 95.31%
--- Epoch 2 done: Loss 0.1635, Acc 95.33%
Epoch 3/5, Batch 200, Loss: 0.0969, Acc: 96.25%
Epoch 3/5, Batch 400, Loss: 0.0294, Acc: 96.52%
Epoch 3/5, Batch 600, Loss: 0.2586, Acc: 96.52%
Epoch 3/5, Batch 800, Loss: 0.0783, Acc: 96.59%
Epoch 3/5, Batch 1000, Loss: 0.0639, Acc: 96.63%
Epoch 3/5, Batch 1200, Loss: 0.0188, Acc: 96.61%
Epoch 3/5, Batch 1400, Loss: 0.0162, Acc: 96.64%
Epoch 3/5, Batch 1600, Loss: 0.0267, Acc: 96.65%
Epoch 3/5, Batch 1800, Loss: 0.0536, Acc: 96.67%
--- Epoch 3 done: Loss 0.1142, Acc 96.69%
Epoch 4/5, Batch 200, Loss: 0.1117, Acc: 97.20%
Epoch 4/5, Batch 400, Loss: 0.0594, Acc: 97.28%
Epoch 4/5, Batch 600, Loss: 0.2765, Acc: 97.29%
Epoch 4/5, Batch 800, Loss: 0.0615, Acc: 97.41%
Epoch 4/5, Batch 1000, Loss: 0.0588, Acc: 97.46%
Epoch 4/5, Batch 1200, Loss: 0.0065, Acc: 97.49%
Epoch 4/5, Batch 1400, Loss: 0.1646, Acc: 97.42%
Epoch 4/5, Batch 1600, Loss: 0.0188, Acc: 97.42%
Epoch 4/5, Batch 1800, Loss: 0.0693, Acc: 97.43%
--- Epoch 4 done: Loss 0.0897, Acc 97.42%
Epoch 5/5, Batch 200, Loss: 0.0449, Acc: 97.84%
Epoch 5/5, Batch 400, Loss: 0.0252, Acc: 97.93%
Epoch 5/5, Batch 600, Loss: 0.2142, Acc: 97.93%
Epoch 5/5, Batch 800, Loss: 0.8323, Acc: 97.84%
Epoch 5/5, Batch 1000, Loss: 0.1653, Acc: 97.80%
Epoch 5/5, Batch 1200, Loss: 0.0093, Acc: 97.84%
Epoch 5/5, Batch 1400, Loss: 0.0633, Acc: 97.85%
Epoch 5/5, Batch 1600, Loss: 0.0891, Acc: 97.83%
Epoch 5/5, Batch 1800, Loss: 0.1162, Acc: 97.83%
--- Epoch 5 done: Loss 0.0733, Acc 97.82%
```

Instantiating and training the network with the debug output

The debug output shows the epoch, batch number, current loss and accuracy. The accuracy goes up fast in the first epoch (72.33% at batch 200 to 89.56% at the end), and the average loss goes from 0.3637 in epoch 1 to 0.0733 in epoch 5. At the end the training accuracy is 97.82%.

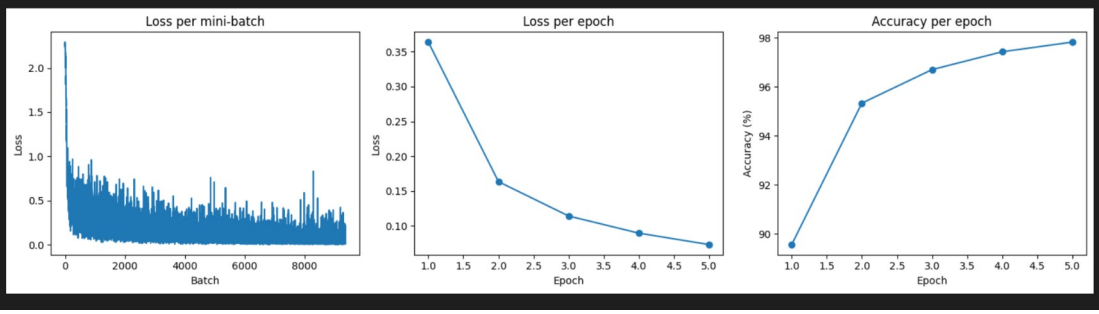
```
[7] # TASK - 6 . 1.
    plt.figure(figsize=(15, 4))

    plt.subplot(1, 3, 1)
    plt.plot(all_losses)
    plt.title("Loss per mini-batch")
    plt.xlabel("Batch")
    plt.ylabel("Loss")

    plt.subplot(1, 3, 2)
    plt.plot(range(1, len(epoch_losses)+1), epoch_losses, marker='o')
    plt.title("Loss per epoch")
    plt.xlabel("Epoch")
    plt.ylabel("Loss")

    plt.subplot(1, 3, 3)
    plt.plot(range(1, len(epoch_accs)+1), epoch_accs, marker='o')
    plt.title("Accuracy per epoch")
    plt.xlabel("Epoch")
    plt.ylabel("Accuracy (%)")

    plt.tight_layout()
    plt.show()
```



Loss of every mini-batch, loss per epoch and accuracy per epoch

The first graph is the loss of every mini-batch that the function returned. It is noisy because every batch is different, but it clearly goes down. The other two graphs show that the loss gets lower and the accuracy gets higher every epoch, and the improvement gets smaller after epoch 3.

Task 7: Testing Network

We tested the network on the whole test set using torch.no_grad(), since we don't need gradients for testing. torch.max(outputs, 1) gives the digit with the highest score for every image. Then we tried the network on a few test images and showed the prediction (P) and the true label (T). Green means correct and red means wrong.

```
[8]  # TASK - 7. Testing Network.

     correct = 0
     total = 0

     with torch.no_grad():
         for inputs, labels in test_loader:
             outputs = net(inputs)
             _, predicted = torch.max(outputs, 1)
             #^^takes the highest score along dimension 1, the digit with the highest score per image.
             total += labels.size(0)
             correct += (predicted == labels).sum().item()

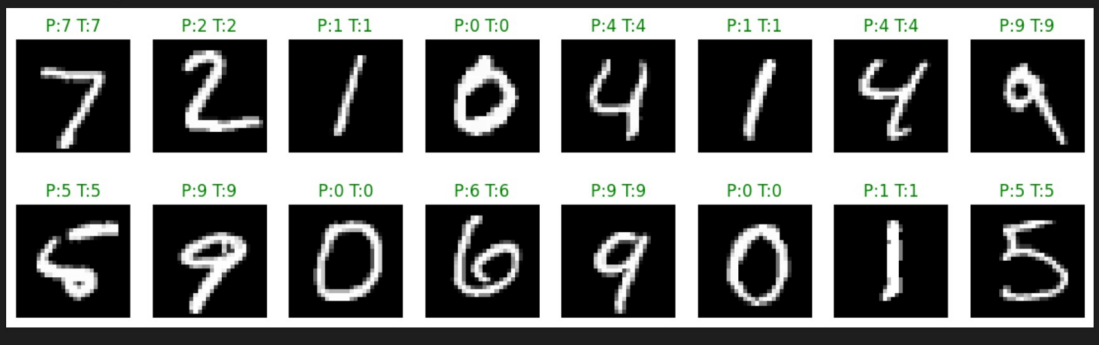
     print(f"Test accuracy: {100*correct/total:.2f}% ({correct}/{total})")

     # Predictions on a few test images
     images, labels = next(iter(test_loader))

     with torch.no_grad():
         outputs = net(images)
         _, predicted = torch.max(outputs, 1)

     fig, axes = plt.subplots(2, 8, figsize=(14, 4))
     for i, ax in enumerate(axes.flat):
         ax.imshow(images[i].squeeze(), cmap='gray')
         color = 'green' if predicted[i] == labels[i] else 'red'
         ax.set_title(f"P:{predicted[i].item()} T:{labels[i].item()}", color=color)
         ax.axis('off')
     plt.show()

✓
Test accuracy: 97.31% (9731/10000)
```



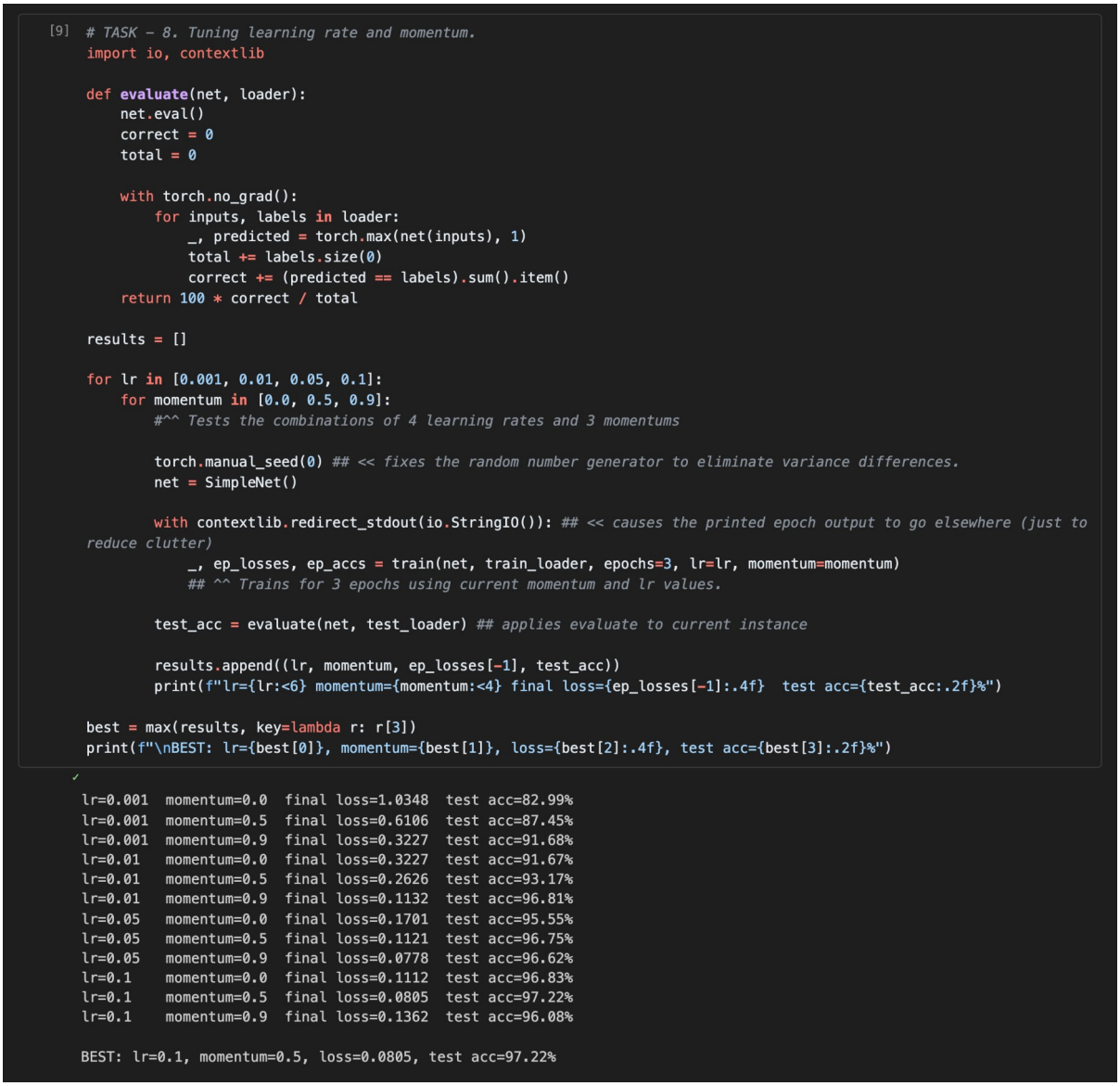
Testing source code and results

The test accuracy is 97.31% (9731 out of 10000). This is very close to the training accuracy (97.82%), so the network works well on images it did not see before. All 16 test images we showed were predicted correctly.

Task 8: Tuning the parameters and model architecture

Learning rate and momentum (TASK - 8 in the code)

We tried 4 learning rates (0.001, 0.01, 0.05, 0.1) and 3 momentum values (0.0, 0.5, 0.9), so 12 combinations. Every combination trains a new SimpleNet for 3 epochs, and we use torch.manual_seed(0) before each one so all of them start from the same weights and the comparison is fair. We hid the training prints to keep the output clean.



```
[9] # TASK – 8. Tuning learning rate and momentum.
    import io, contextlib

    def evaluate(net, loader):
        net.eval()
        correct = 0
        total = 0

        with torch.no_grad():
            for inputs, labels in loader:
                _, predicted = torch.max(net(inputs), 1)
                total += labels.size(0)
                correct += (predicted == labels).sum().item()
        return 100 * correct / total

    results = []

    for lr in [0.001, 0.01, 0.05, 0.1]:
        for momentum in [0.0, 0.5, 0.9]:
            #^^ Tests the combinations of 4 learning rates and 3 momentums

            torch.manual_seed(0) ## << fixes the random number generator to eliminate variance differences.
            net = SimpleNet()

            with contextlib.redirect_stdout(io.StringIO()): ## << causes the printed epoch output to go elsewhere (just to
    reduce clutter)
                _, ep_losses, ep_accs = train(net, train_loader, epochs=3, lr=lr, momentum=momentum)
                ## ^^ Trains for 3 epochs using current momentum and lr values.

            test_acc = evaluate(net, test_loader) ## applies evaluate to current instance

            results.append((lr, momentum, ep_losses[-1], test_acc))
            print(f"lr={lr:<6} momentum={momentum:<4} final loss={ep_losses[-1]:.4f}  test acc={test_acc:.2f}%")

    best = max(results, key=lambda r: r[3])
    print(f"\nBEST: lr={best[0]}, momentum={best[1]}, loss={best[2]:.4f}, test acc={best[3]:.2f}%")
```

```
lr=0.001  momentum=0.0  final loss=1.0348  test acc=82.99%
lr=0.001  momentum=0.5  final loss=0.6106  test acc=87.45%
lr=0.001  momentum=0.9  final loss=0.3227  test acc=91.68%
lr=0.01   momentum=0.0  final loss=0.3227  test acc=91.67%
lr=0.01   momentum=0.5  final loss=0.2626  test acc=93.17%
lr=0.01   momentum=0.9  final loss=0.1132  test acc=96.81%
lr=0.05   momentum=0.0  final loss=0.1701  test acc=95.55%
lr=0.05   momentum=0.5  final loss=0.1121  test acc=96.75%
lr=0.05   momentum=0.9  final loss=0.0778  test acc=96.62%
lr=0.1    momentum=0.0  final loss=0.1112  test acc=96.83%
lr=0.1    momentum=0.5  final loss=0.0805  test acc=97.22%
lr=0.1    momentum=0.9  final loss=0.1362  test acc=96.08%

BEST: lr=0.1, momentum=0.5, loss=0.0805, test acc=97.22%
```

Tuning learning rate and momentum

Best result: learning rate 0.1 with momentum 0.5, final loss 0.0805 and test accuracy 97.22%.

With a small learning rate (0.001) and no momentum the network learns very slow (82.99%). Adding momentum helps a lot when the learning rate is small. But when both are big (lr 0.1 and momentum 0.9) the result gets worse (96.08%) because the steps are too big. Also lr 0.001 with momentum 0.9 gave almost the same result as lr 0.01 with no momentum, so momentum works like making the learning rate bigger.

Number of layers and number of neurons (TASK - 8 . 1 in the code)

To change the architecture we made FlexNet, which takes a list of hidden layer sizes, so [512, 256] means 2 hidden layers with 512 and 256 neurons. We tried 7 architectures with 1, 2 and 3 hidden layers. We used the best lr and momentum from the first part and trained each one for 3 epochs with the same seed.

```
[10]  # TASK – 8 . 1. Tuning the number of layers and neurons.

      class FlexNet(nn.Module):
          def __init__(self, hidden_sizes, input_size=784, num_classes=10):
              super().__init__()
              self.hidden = nn.ModuleList()
              ## ^^ holds the hidden layers, one nn.Linear for every value in hidden_sizes.
              for h in hidden_sizes:
                  self.hidden.append(nn.Linear(input_size, h))
                  input_size = h
              self.out = nn.Linear(input_size, num_classes)

          def forward(self, x):
              x = torch.flatten(x, 1)
              for layer in self.hidden:
                  x = F.relu(layer(x))   # ReLU after every hidden layer
              return self.out(x)

      arch_results = []

      for hidden in [[64], [128], [256], [512], [256, 128], [512, 256], [512, 256, 128]]:
          #^^ 1, 2 and 3 hidden layers with different numbers of neurons

          torch.manual_seed(0)
          net = FlexNet(hidden)

          with contextlib.redirect_stdout(io.StringIO()):
              _, ep_losses, ep_accs = train(net, train_loader, epochs=3, lr=best[0], momentum=best[1])
              ## ^^ uses the best lr and momentum from TASK – 8.

          test_acc = evaluate(net, test_loader)

          arch_results.append((hidden, ep_losses[-1], test_acc))
          print(f"layers={len(hidden)}  neurons={str(hidden):<16} final loss={ep_losses[-1]:.4f}  test acc={test_acc:.2f}%")

      best_arch = max(arch_results, key=lambda r: r[2])
      print(f"\nBEST: layers={len(best_arch[0])}, neurons={best_arch[0]}, loss={best_arch[1]:.4f}, test acc=
      {best_arch[2]:.2f}%")

      ✓
      layers=1  neurons=[64]             final loss=0.0977  test acc=96.80%
      layers=1  neurons=[128]            final loss=0.0805  test acc=97.22%
      layers=1  neurons=[256]            final loss=0.0749  test acc=97.13%
      layers=1  neurons=[512]            final loss=0.0699  test acc=97.68%
      layers=2  neurons=[256, 128]       final loss=0.0697  test acc=96.39%
      layers=2  neurons=[512, 256]       final loss=0.0614  test acc=97.89%
      layers=3  neurons=[512, 256, 128]  final loss=0.0685  test acc=97.18%

      BEST: layers=2, neurons=[512, 256], loss=0.0614, test acc=97.89%
```

Tuning the number of layers and neurons

Best result: 2 hidden layers with 512 and 256 neurons, final loss 0.0614 and test accuracy 97.89%. This is better than the simple network in the first part (97.22%).

With 1 hidden layer, more neurons was better in general (64 neurons gave 96.80% and 512 gave 97.68%). More layers did not always help. [256, 128] was worse than 1 layer, and 3 layers (97.18%) was worse than 2 layers. So a bigger network can do better, but only up to a point. MNIST is simple, and with only 3 epochs the deeper network does not have enough time to learn.

Task 9: Save your source code

All the source code is saved in the notebook "Assigment-1 DL.ipynb" and uploaded to GitHub.

GitHub link: https://github.com/HussainNabi/Deep-Learning